

SmartLock

Présentation du projet :

- *Le projet SmartLock modernise la gestion du matériel du DeVinci Fablab en remplaçant les serrures traditionnelles par un système d'accès sécurisé et connecté via badge NFC.*
- *Cette solution couple la sécurisation physique des armoires à une gestion centralisée des stocks pour garantir une traçabilité complète des emprunts en temps réel.*
- *A terme, la solution sera implémentée toutes les armoires du Fablab afin de sécuriser le matériel de l'espace.*

VI - VF	Guide de maintenance	06/06/2026
Rédacteur : Florian Cazac	Relecteurs : Augustin Winterbert, Alexandre Houdin, Guilhem Henoux, Lois Halmaert, Yann Videmment	Jalon tests Systeme
	Validateurs : Florian Cazac	





1. Table des matières

2.	Introduction	2
2.1.	Prérequis	2
3.	Architecture du système.....	2
4.	Inventaire matériel.....	2
4.1.	Brochage — Lecteur NFC PN532 ↔ Raspberry Pi (I ² C).....	2
4.2.	Brochage — Relais solénoïde ↔ Raspberry Pi.....	3
5.	4. Déploiement et configuration	3
5.1.	IHM — Raspberry Pi Zero 2W	3
5.2.	API d'authentification — Serveur Docker.....	3
5.3.	Dashboard — Serveur Docker.....	4
6.	Configuration Keycloak.....	4
6.1.	Clients Keycloak	4
6.2.	Rôles système.....	4
7.	Logs et surveillance	5
8.	Guide de dépannage.....	5
9.	Procédures de mise à jour	6
9.1.	IHM (Raspberry Pi)	6
9.2.	API	6
9.3.	Dashboard.....	6
10.	Limitations connues et évolutions futures	6



2. Introduction

Ce guide est destiné à l'équipe technique du De Vinci FabLab et aux successeurs du projet SmartLock. Il décrit l'architecture, les procédures de déploiement, de configuration, de surveillance et de dépannage.

2.1. Prérequis

- Accès SSH au Raspberry Pi Zero 2W (réseau local ou VPN)
- Accès au serveur hébergeant l'API et le Dashboard (Docker + SSH)
- Accès administrateur à l'instance Keycloak (<https://auth.devinci-fablab.fr>)
- Outils : git, docker, docker compose, curl, jq, multimètre

3. Architecture du système

Composant	Technologie	Hébergement	URL / Accès
IHM	Python 3.13 / CustomTkinter	Raspberry Pi Zero 2W	SSH locker@<ip>
API Auth	FastAPI + PostgreSQL	Serveur De Vinci (Docker)	https://api.smartlock.devinci-fablab.fr
Dashboard	SvelteKit / Node.js	Serveur De Vinci (Docker)	https://dashboard.smartlock.devinci-fablab.fr
Keycloak	Keycloak 25+	Serveur De Vinci (Docker)	https://auth.devinci-fablab.fr

4. Inventaire matériel

Composant	Spécification	Rôle
Raspberry Pi Zero 2W	ARM Cortex-A53 1GHz, 512 MB RAM, WiFi 802.11n	Processeur principal, exécute l'IHM Python, contrôle GPIO
Solénoïde 12V	Push-pull 12V DC, ~0.5A, relais 5V sur GPIO 26 (BCM)	Mécanisme de verrouillage physique
Alimentation	12V 3A (36W), jack 5.5/2.1mm	Source d'énergie principale
Écran tactile 7"	Waveshare 7inch HDMI LCD (H), 1024×600, capacitif 5pts	Interface kiosque, HDMI + USB touch
Convertisseur buck	DC-DC 12V → 5V, 3A max	Alimentation RPi et écran depuis 12V
Lecteur NFC PN532	I ² C / SPI, 13.56 MHz, adresse I ² C 0x24	Lecture des badges NFC utilisateurs

4.1. Brochage — Lecteur NFC PN532 ↔ Raspberry Pi (I²C)

Broche PN532	Broche RPi (nom)	Pin physique
VCC	3.3V	Pin 1
GND	GND	Pin 6
SDA	GPIO 2 / SDA	Pin 3
SCL	GPIO 3 / SCL	Pin 5

Vérification : `i2cdetect -y 1` → doit afficher l'adresse 0x24.



4.2. Brochage — Relais solénoïde ↔ Raspberry Pi

Broche Relais	Broche RPi (nom)	Pin physique
IN (signal)	GPIO 26 (BCM)	Pin 37
VCC	5V	Pin 2
GND	GND	Pin 6

Note : GPIO 27 (Pin 13) réservé pour le capteur Hall — non câblé en version actuelle.

5.4. Déploiement et configuration

5.1. IHM — Raspberry Pi Zero 2W

Dépôt : SmartLock-IHM

Installation des dépendances système

```
sudo apt-get update && sudo apt-get install -y python3-pip python3-tk xorg xinput
i2c-tools
pip install customtkinter python-dotenv requests Pillow
pip install adafruit-blinka adafruit-circuitpython-pn532 RPi.GPIO
```

Configuration de l'environnement

```
cp .env.example .env
# Renseigner : API_URL, KEYCLOAK_URL, KEYCLOAK_REALM, LOCKER_CLIENT_SECRET,
LOCKER_ID
# Mettre SIMULATION_MODE=False pour la production
```

Activation I²C

```
sudo raspi-config → Interface Options → I2C → Enable
i2cdetect -y 1 # Vérifier adresse 0x24
```

Auto-start au boot (~/.xinitrc)

```
# Copier le fichier .xinitrc du dépôt vers /home/locker/.xinitrc
# Il configure : rotation portrait 90°, calibration touch, boucle de redémarrage
Python
```

5.2. API d'authentification — Serveur Docker

Dépôt : SmartLock-Authentication-Authorization

```
cp .env.example .env # Renseigner DATABASE_URL, KEYCLOAK_URL, SECRET_KEY
docker compose up -d --build
docker compose exec api alembic upgrade head # Appliquer les migrations
curl https://api.smartlock.devinci-fablab.fr/health # → {"status":"ok"}
```



5.3. Dashboard — Serveur Docker

Dépôt : SmartLock-Dashboard

```
cd docker
cp web/.env.example web/.env # Voir variables ci-dessous
docker compose -f compose.prod.yaml up -d --build
curl http://localhost:3000/health # → {"status":"ok"}
```

Variables Dashboard requises :

```
PUBLIC_SMARTLOCK_API_URL=https://api.smartlock.devinci-fablab.fr
KEYCLOAK_ISSUER=https://auth.devinci-fablab.fr/realms/dev
KEYCLOAK_CLIENT_ID=smartlock-dashboard
KEYCLOAK_CLIENT_SECRET=<depuis-admin-keycloak>
KEYCLOAK_REDIRECT_URI=https://dashboard.smartlock.devinci-fablab.fr/login/callback
SESSION_SECRET=<openssl rand -hex 32>
```

6. Configuration Keycloak

URL : <https://auth.devinci-fablab.fr> — Realm : **dev** (test) ou **prod** (production)

6.1. Clients Keycloak

Client ID	Type	Accès autorisé
smartlock-dashboard	OIDC public (PKCE)	Toutes les routes Dashboard
smartlock-api	Client credentials	Proxy Admin Keycloak (manage-users)
smartlock-lockers	Client credentials	POST /auth/locker/{id}/check uniquement
nfc-scanner	Client credentials	POST /badge/scan uniquement

6.2. Rôles système

Rôle	Niveau	Description
Admin	T5	Accès complet — configuration infrastructure
codir / présidence	T4	Direction — accès audit complet
materialiste	T3	Responsable du matériel
electronique / textile / 3d	T2	Responsables de domaine
Bureau	T2	Bureau de l'association
membre	T0	Accès de base aux armoires autorisées



7. Logs et surveillance

Source	Commande / Chemin	Notes
IHM (Raspberry Pi)	~/smartlock.log	Truncation auto à 5 MB. Contient : tokens, NFC, GPIO, transactions.
API (Docker)	docker compose logs api	Logs FastAPI/Uvicorn. Filtrer : docker compose logs api grep ERROR
Dashboard (Docker)	docker compose logs dashboard	Logs SvelteKit/Node. Erreurs de session et d'auth.
Audit métier	GET /access_log/ ou Dashboard /logs	Immuable — ne pas supprimer les entrées.

8. Guide de dépannage

Problème	Cause probable	Solution
IHM ne démarre pas au boot	Problème X11 / xinitrc	Vérifier ~/.xinitrc, relancer : startx &
Badge non détecté	I²C mal configuré ou PN532 absent	i2cdetect -y 1 → vérifier 0x24. Contrôler câblage SDA/SCL.
API répond 401 / 403	Token expiré ou secret incorrect	Vérifier LOCKER_CLIENT_SECRET dans .env. Renouveler dans Keycloak admin.
Solénoïde n'actionne pas	GPIO 26 inactif ou relais défaillant	python -c "import RPi.GPIO as G; G.setmode(G.BCM); G.setup(26,G.OUT); G.output(26,1)"
Écran noir après démarrage	Tension U2 insuffisante (<4.8V)	Mesurer sortie buck. Vérifier connexion HDMI.
Dashboard inaccessible	KEYCLOAK_ISSUER mal configuré	Vérifier .env. Tester GET /health → {"status":"ok"}
Erreur alembic upgrade head	Migration déjà appliquée ou conflit DB	docker compose exec api alembic current
Log ~/smartlock.log trop gros	Pas de rotation automatique	Truncation auto à 5 MB intégrée. Si besoin : truncate -s 0 ~/smartlock.log



9. Procédures de mise à jour

9.1. IHM (Raspberry Pi)

```
ssh locker@<adresse-ip-rpi>
cd SmartLock-IHM && git pull origin main
pip install -r requirements.txt --upgrade
sudo reboot    # ou pkill -f main.py pour redémarrer sans reboot
```

9.2. API

```
cd SmartLock-Authentication-Authorization && git pull origin main
docker compose build api && docker compose up -d api
docker compose exec api alembic upgrade head    # si nouvelles migrations
```

9.3. Dashboard

```
cd SmartLock-Dashboard/docker && git pull origin main
docker compose -f compose.prod.yaml build dashboard
docker compose -f compose.prod.yaml up -d dashboard
```

10. Limitations connues et évolutions futures

Batterie de secours : Livraison des R_SC → intégration PCB → validation BMS → test autonomie 30 min

Capteur effet Hall : Câbler GPIO 27 (code déjà présent) → valider SYS-10

SKSM : Définir les spécifications mécaniques → implémenter en version future

Notifications Discord : Endpoint /notifications/discord prévu dans l'IHM mais non connecté côté API